TDS6213 Data Structures and Algorithms

Trimester October 2024

Pharmacy Inventory Management System

Group 2

Lab Section: 2A

Project Prepared by:

	Name	Student ID	Major
1	Cheong Kai Jin	241UT240K7	AI
2	Kang Xuan Xin	241UT240W1	AI
3	Chong Zheng Cheng	241UT240LQ	AI
4	Huda Abdalla Elbashir Elmahdi	243UT243N4	AI

FACULTY OF INFORMATION SCIENCE & TECHNOLOGY

MULTIMEDIA UNIVERSITY

MALAYSIA

25/1/2026

Table of Contents

INTRODUCTION 3
PROBLEM STATEMENTS 4
OBJECTIVES 5
IMPLEMENTATION DETAILS 6
Data Structure 6
Algorithms 8
ADT SPECIFICATIONS 12
PROGRAM SCREENSHOT 17
CONCLUSION 26
TASK DECLARATION FORM 27

INTRODUCTION

The Pharmacy Inventory Management System was created to help pharmacies handle their medicine stock in a more organized, accurate, and efficient way. A lot of pharmacies still use manual record-keeping or old systems, which raises the chance of mistakes. This can also cause inconsistent data and poor inventory control, possibly causing too much stock, shortages, and expired medicines.

This project uses Data Structures and Algorithms concepts with C++ to create a system for managing inventory. It handles key pharmacy tasks like adding, deleting, viewing, finding, and sorting medicine records. The goal is that employing proper data structures and algorithms will make data management quicker, improve information retrieval, and improve pharmacy workflow.

Working on this project allows for practical experience in building data structures independently, rather than depending on pre-made tools. This strengthens understanding of how algorithms function in applied settings.

PROBLEM STATEMENTS

1- Many pharmacies depend on either paper records or old computer systems, which can lead to mistakes and, in turn, wrong inventory counts.

2- Bad inventory tracking can cause too much or too little stock, costing the pharmacy money and making customers unhappy.

3- If not watched properly, drugs that are past their expiration date may sit on shelves, which threatens patient safety and breaks rules.

4- Pharmacies without a good system have a hard time getting current stock numbers, making it tough to know which medicines are selling fast or slow.

5- Without well-organized inventory information, pharmacy workers and bosses can't easily make reports, keep track of things, or decide what to do.

OBJECTIVES

The objective of this project is to develop a Pharmacy Inventory Management System using C++. The purpose is to enhance the accuracy and organization of medicine stock record keeping.

The system will do the following:

1- Maintaining a simple log of all medications, noting their names, amounts, and expiration dates, helps keep records structured and lowers the chance of errors when monitoring inventory.

2- The system is designed to monitor and revise inventory records as stock levels are modified through receipts, sales, or disposals. This process seeks to maintain data accuracy, reduce mistakes, and improve overall accountability.

3- Incorporate search and sort functions for swift data retrieval, allowing for informed de cisions and improved service

4- Create basic pharmacy inventory reports for example, daily or weekly stock use, near-expiry drugs, and low stock levels. This helps managers with planning, cost control, and safety regulation compliance.

IMPLEMENTATION DETAILS

Data Structure

Linked List

Linked List is the data structure used to store both the main list of medicinal items and the individual batches associated with each item. This structure consists of individual elements called Nodes, where each node contains the data (such as an Item or a Batch) and a pointer that is linked to the next node in the sequence. It is used because of its dynamic memory allocation capabilities. Unlike a normal array or a hashmap, which has a fixed size determined at the beginning of the program, a linked list may grow or shrink as elements are added or removed during daily operation of the store. This is ideal for pharmacy inventory management system which involves frequent insertion of new items and deletions of expiry items without wasting computer memory.

Node Implementation (Struct)

Member	Type	Description
data	T (template type)	The data of the current node.
next	Node*	Pointer to next node.
Node	Constructor	Constructor for Node.

Linked List Implementation

Member	Type	Description
head	Node*	Pointer to the first node in list.
size	int	The number of nodes in the list.
LinkedList	Constructor	Constructor for LinkedList.
~LinkedList	Desctructor	Destructor for LinkedList, frees all allocated memory for nodes.
get_size	Method	Returns the size of the list.
get_node	Method	Returns reference to the node at given index.
insert_node	Method	Inserts a new node with given data at given index.
delete_node	Method	Deletes a node at given index.

Queue

Besides, the system also uses Queue to keep track of all the inventory transactions, which include sales, refunds, restocks, and discards. This Queue is implemented as a wrapper around the Linked List, which uses the same dynamics growing capabilities of Linked List and enforces the logical order of First-In, First-Out (FIFO). This exact logic was selected as it is the standard accounting practices of a real-world business, where the transaction history must be maintained in the same sequence as they happened. When the user views the

transaction list, the Queue ensures that the oldest records are processed first and the newest records can be appended to the end, providing a clear and reliable audit trail. Compared to Stack, Queue supports front and end access, which is better for processing in future features.

Queue Implementation

Member	**Type**	**Description**
head	Node*	Pointer to the first node in list.
tail	Node*	Pointer to the last node in list.
size	int	The number of nodes in the list.
Queue	Constructor	Constructor for Queue.
get_size	Method	Returns the size of the list.
get_front	Method	Returns the data of first node.
get_back	Method	Returns the data of last node.
get_index	Method	Returns reference to the node at given index.
enqueue	Method	Inserts a new node at the back of queue.
dequeue	Method	Deletes a node at the front of queue.

Algorithms

Linear Search

The searching algorithm for our project is linear search, also known as sequential search. It is a simple searching technique that checks elements one by one from the beginning of the structure. Each element is compared with the target element; search will stop immediately when the element is matched. If current element is not matching with target, move to next elements. The process is repeated until it finds a match or reaches the end.

The reason that we selected this algorithm is because we are using linked list as our data structure. In linked list, each node will only know its own data and reference to next node, they were unable to know the other node's information. It does not provide direct access as array to get the elements, the traversal must start from the head of the linked list. In this case, linear search matches this traversal behavior and be the appropriate searching technique for our data structure. Besides, linear search is efficient for our system that use a small dataset (roughly 100 items), and the search will happen instantly, thus making complex algorithms unnecessary.

```
int Item::find_batch(std::string batch_id) {
    // linear search
    for (int i = 0; i < item_batches.get_size(); ++i) {
        Batch* b = &item_batches.get_node(i)->data;
        if (b && b->get_batch_id() == batch_id) return i;
    }
    return -1;
}
```

```
int Inventory::find_item(std::string item_id) {
    // linear search
    for (int i = 0; i < items.get_size(); ++i) {
        if (get_item(i)->get_item_id() == item_id) return i;
    }
    return -1;
}
```

Code screeshot taken from adt.cpp

In the project implementation, when Item::find_batch function is called, it performs linear search on the list of batches with a specific item (based on user input: batch_id). The function uses a for loop to traverse the batch linked list and the loop continues as long as counter i less than the total item_batch's store in item. During each iteration, function will access the current batch node's and compare the batch_id with the user input batch_id. If the ID is match, function will return the index i that indicate the matches batch has been found. Otherwise, it will continue to next batch node until end of the item's batch. If still not found the target batch_id until the end of batch, function will return -1 that mean the target batch-id is no exist in this item's batch.

Similarly, function Inventory::find_item also applies linear search as searching algorithm. When the function is invoked, it uses loops to execute linear search on inventory item list.

The search starts from the first element and visits sequentially each element until finds the item_id. The loop condition is checking the counter i less than the total item_list's size to ensure each item stored in the inventory can be retrieve. Function will access the current item and retrieve its item_id to compare with target item_id. If match, function will immediate returns i which represent the index of the found item. Otherwise, return -1 that indicates the target item_id cannot be found in inventory.

Selection Sort

```cpp

void command_list_items(Inventory& inventory) {
    // sort item_id in alphabetical ascending order using selection sort
    int n = inventory.get_items_size();
    Item* sorted_items[n];
    // copy LinkedList items into sorted_items array
    for (int i = 0; i < n; ++i) {
        sorted_items[i] = inventory.get_item(i);
    }
    // selection sort
    for (int i = 0; i < n - 1; ++i) {
        int min_idx = i;
        for (int j = i + 1; j < n; ++j) {
            if (sorted_items[j]->get_item_id() < sorted_items[min_idx]->get_item_id())
                min_idx = j;
        }
        std::swap(sorted_items[i], sorted_items[min_idx]);
    }
    // list all items in inventory
    std::cout << "\nItems are listed below as item ID (ascending) and total stock quantity:\n";
    for (int i = 0; i < n; ++i) {
        std::cout << "\n"
            << sorted_items[i]->get_item_id() << " - quantity: "
            << sorted_items[i]->get_total_quantity();
    }
}

```

Code screeshot taken from cmd.cpp

The *command_list_items* function utilizes the Selection Sort algorithm on the inventory data to sort the data alphabetically before it is displaying to the user. Since the original inventory is stored in a LinkedList, which is difficult to sort directly without complex pointer manipulation, the function first copies the memory addresses of each Item into a temporary array. This allows the algorithm to rearrange the items in a virtual environment without risking the integrity of the actual data stored in the inventory.

Sorting starts with an outer loop going through each element in the temporary array from the first one to the last one. At each position, the algorithm considers the element currently in that position is the "minimum" (the one which should come first in alphabetic order). The inner loop then scans the rest of the unsorted array to find an item with an ID that is actually "less" or comes earlier in the alphabet than the current minimum. This check is done with the string ID of the product, for example: "bandaid" is checked against "panadol", "bandaid" comes first.

After the inner loop has completed traversing the rest of the list and found the actual smallest item for that position, the algorithm will then perform a swap. It will exchange the item at the current index of the loop with the item at the minimum index, to ensure the correct product is placed in its proper alphabetical place. This process of finding the minimum and swapping repeats until the entire array is sorted from A to Z. Once the sorting is done, function goes through the sorted temporary array for the final time to print each item's ID and the total stock quantity to the console.

Selection sort was used in the *command_list_items* function because it is easy to understand and it clearly shows fundamental programming logic in a readable professional way. Selection sort sorts the data by swapping the elements within the existing structure. It does not need to create a whole new copy of the list for doing the sort like how merge sort did. Advanced

algorithms, such as Merge Sort or Quick Sort, provide faster speeds for large amounts of data, but they tend to be more complex, and they often involve recursive logic and require much more memory. For this Pharmacy Inventory Management System, the $O(n^2)$ time complexity of Selection Sort is acceptable because the expected inventory size is small (around 100 items), which does not incur a significant performance difference if use other advanced sorting algorithm.

ADT SPECIFICATIONS

Brief on the changes:

- Added a new ADT called "Inventory" because we realized that we didn't have a global container for Item objects and Transaction objects. Its attributes and operation are added alongside.
- Added constructors for all ADTs.
- Added more getters and setters.
- Added and changed some attributes to support a better system structure.

Final ADT Specification

ADT Name	Category	Member	Type	Description	Changes / Remarks
Date	Attributes	day	int	Stores the day of the date.	-
		month	int	Stores the month of the date.	-
		year	int	Stores the year of the date.	-
	Operations	Date	Constructor	Contructs a Date object using the provided parameters.	Added for the construction of the Date object.
		get_day	Method	Returns the value of member "day".	-
		get_month	Method	Returns the value of member "month".	-
		get_year	Method	Returns the value of member "year".	-
		is_after	Method	Compares this date object with another provided otherDate object, returns true if this date object is AFTER otherDate, else false.	Renamed from Is After Date.
		is_valid	Method	Check if this date is a real and valid date that exists.	Renamed from Is Valid Date.
Batch	Attributes	batch_id	string	Stores the unique ID of an item batch.	Changed from integer value to string to support a better ID system.
		batch_description	string	Stores the description of the item batch.	Added for information.
		added_date	Date	Stores the date when the batch is added.	Added for information.
		expiry_date	Date	Stores the expiry date of the batch.	No longer acts as a unique identifier,

					replaced by batch ID.
		batch_stock	int	Stores the remaining stock quantity of the batch.	Renamed from Item Quantity.
	Operations	Batch	Constructor	Contructs a Batch object using the provided parameters.	Added for the construction of the Batch object.
		get_batch_id	Method	Returns the value of member “batch_id”.	Added for access.
		get_batch_description	Method	Returns the value of member “batch_descrption”.	Added for access.
		get_added_date	Method	Returns the value of member “added_date”.	Renamed from Get Date.
		get_expiry_date	Method	Returns the value of member “expiry_date”.	Added for access.
		get_batch_stock	Method	Returns the value of member “batch_stock”.	Renamed from Get Item Quantity.
		set_batch_description	Method	Sets the value of member “batch_description”.	Added for access.
		set_batch_stock	Method	Sets the value of member “batch_stock”.	Added for access.
		is_expired	Method	Checks if the batch expiry date is after today’s date.	Renamed from Check Expired.
Item	Attributes	item_id	string	Stores the unique ID for each item.	-
		item_description	string	Stores the description for each item.	-
		batch_record	int	Stores the historical count of added batches, used for generating unique batch IDs.	Added for unique batch ID generation.
		item_batches	LinkedList< Batch>	A list of Batch objects that belong to this item.	-
	Operations	Item	Constructor	Contructs an Item object using the provided parameters.	Added for the construction of the Item object.
		get_item_id	Method	Returns the value of member “item_id”.	-
		get_item_description	Method	Returns the value of member “item_description”.	-
		get_batch_record	Method	Returns the value of member “batch_record”.	Added for access.
		get_batches_size	Method	Returns the size of member “item_batches”.	Added for access.
		get_total_quantity	Method	Calculates and returns the sum of quantity from all batches that belong to this item.	-

		get_expired_quantity	Method	Calculates and returns the sum of quantity from expired batches that belong to this item.	Added for access.
		get_batch	Method	Returns the reference to the Batch object at given index in the list “item_batches”.	-
		set_item_description	Method	Sets the value of member “item_description”.	Renamed from Update Description.
		add_batch	Method	Adds a new batch object with the provided parameters into the list “item_batches”.	-
		find_batch	Method	Finds the index of a specific batch with the given batch ID.	Added for finding batch index by batch ID, used in command_edit_batch_desc and more.
		remove_batch	Method	Removes a Batch object from list “item_batches” by index.	-
Transaction	Attributes	transaction_id	string	Stores the unique ID for the transaction.	Added as the unique identifier for Transaction objects.
		item_id	string	Stores the Item ID of the item in the transaction.	-
		batch_id	string	Stores the Batch ID of the item batch in the transaction.	Added for information.
		transaction_date	Date	Stores the date when the transaction is made.	-
		transaction_type	TransactionType	An Enumerator that ranges from 0-3. 0 = SOLD, 1 = REFUND, 2 = RESTOCK, 3 = DISCARD. Stores the type of transaction.	Replaces Transaction Description.
		stock_change	int	Stores the change in stock of the transaction.	Renamed from Item Quantity.
	Operations	Transaction	Constructor	Contructs an Transaction object using the provided parameters.	Added for the construction of the Transaction object.
		get_transaction_id	Method	Returns the value of member “transaction_id”.	-
		get_item_id	Method	Returns the value of member “transaction_id”.	-

		get_batch_id	Method	Returns the value of member “transaction_id”.	Added for access.
		get_transaction_date	Method	Returns the value of member “transaction_id”.	-
		get_transaction_type	Method	Returns the value of member “transaction_type”.	Replaces Get Transaction Description.
		get_stock_change	Method	Returns the value of member “stock_change”.	Renamed from Get Item Quantity.
Inventory	Attributes	items	LinkedList<Item>	A list of Item objects that is added by the user.	Added for Item object storage.
		transactions	Queue<Transaction>	A queue of Transaction objects generated by the system.	Added for Transaction object storage.
		transaction_record	int	Stores the historical count of added transactions, used for generating unique transaction IDs.	Added for unique transaction ID generation.
	Operations	get_items_size	Method	Stores the size of the list “items”.	Added for access.
		get_transactions_size	Method	Stores the size of the queue “transactions”.	Added for access.
		get_transaction_record	Method	Returns the value of member “transaction record”.	Added for access.
		get_item	Method	Returns the reference to the Item object at given index in the list “items”.	Added for access.
		get_transaction	Method	Returns the reference to the Transaction object at given index in the list “transactions”.	Added for access.
		set_transaction_record	Method	Sets the value of member “transaction_record”.	Added for access.
		add_item	Method	Adds a new Item object with the provided parameters into the list “item_batches”.	Added for access.
		find_item	Method	Finds the index of a specific item with the given item ID.	Added for access.
		remove_item	Method	Removes an Item object from list “items” by index.	Added for access.
		add_transaction	Method	Adds a new Transaction object with the provided parameters into the queue “transactions”.	Added for access.
		find_transaction	Method	Finds the index of a specific transaction with the given transaction ID.	Added for access.

Removed Members

ADT Name	Removed Member	Type	Reason
Date	Update Date	Function	Dates mostly stay the same after they are created, so the constructor is well enough for this purpose.
Transaction	Transaction Description	string	Transaction Description was originally used to indicate transaction details, but we realized that its not necessary and replaced it with the better “transaction_type”.
	Get Transaction Description	Function	Removed alongside the member Transaction Description since it is no longer needed.

Changed Features:

We chose to develop a simpler report generation feature with just sales and stock status because the development was taking too much time.

PROGRAM SCREENSHOT

```
++=========================================================++
||                                                         ||
||                       Welcome to                        ||
||                                                         ||
||     ooooooooo.   ooooo ooo        ooooo  .oooooo..o     ||
||     `888   `Y88. `2A2' `88.       .888' d8P'    `Y8     ||
||      888   .d88'  CKJ   888b     d'888  Y88bo.          ||
||      888ooo88P'   KXX   8 Y88. .P  888   `*Y8888o.      ||
||      888          CZC   8  `888'   888       `*Y88b     ||
||      888          HAE   8    Y     888  oo     .d8P     ||
||     o888o        o888o o8o        o888o 8'*88888P'     ||
||                                                         ||
||        [ Pharmacy Inventory Management System ]         ||
||           developed by TDS6213 Lab2A Group 2            ||
||                                                         ||
++=========================================================++


Enter password:
> |
```

This image shows our program welcome page and requires users to enter password (**admin123**).

```
Enter password:
> admin123

Inventory read from: pims.inventory

==========================[ Help Menu ]============================

How to use:
    This application is command based,
    run actions by typing commands in the console.
    The commands are self explanatory.
    More intructions will appear after entering a command.

Commands list:
- additem <item id> <item description>
- addbatch <item id> <stock amount> <expiry date> <batch description>
- edititemdesc <item id> <new item description>
- editbatchdesc <item id> <batch id> <new batch description>
- editbatchstock <item id> <batch id> <transaction type> <change amount>
- deleteitem <item id>
- deletebatch <item id> <batch id>
- listitem
- listbatch <item id>
- listtransaction
- viewitem <item id>
- viewbatch <item id> <batch id>
- report
- save
- load
- exit/end
Commands can be shortened, for example:
- additem -> ai
- deleteitem -> di
- viewbatch -> vb

========================[ End of Menu ]==========================


Enter a command, type "help" for more info.
```

```
Enter a command, type "help" for more info.
> help


==========================[ Help Menu ]============================

How to use:
    This application is command based,
    run actions by typing commands in the console.
    The commands are self explanatory.
    More intructions will appear after entering a command.

Commands list:
- additem <item id> <item description>
- addbatch <item id> <stock amount> <expiry date> <batch description>
- edititemdesc <item id> <new item description>
- editbatchdesc <item id> <batch id> <new batch description>
- editbatchstock <item id> <batch id> <transaction type> <change amount>
- deleteitem <item id>
- deletebatch <item id> <batch id>
- listitem
- listbatch <item id>
- listtransaction
- viewitem <item id>
- viewbatch <item id> <batch id>
- report
- save
- load
- exit/end
Commands can be shortened, for example:
- additem -> ai
- deleteitem -> di
- viewbatch -> vb

========================[ End of Menu ]==========================


Enter a command, type "help" for more info.
> |
```

This image shows help menu. It will always display after user **login successfully** or user can view it by entering **help / h** command.

```
Enter a command, type "help" for more info.
> listitem

Items are listed below as item ID (ascending) and total stock quantity:

aptx-4869 - quantity: 112
bandaid - quantity: 131
centrum - quantity: 124
dettol - quantity: 130
gatorade - quantity: 168
huggies - quantity: 126
oralb - quantity: 126
panadol - quantity: 210
tampax - quantity: 126
vicks - quantity: 84
yakult - quantity: 210
```

```
Enter a command, type "help" for more info.
> listitem

Items are listed below as item ID (ascending) and total stock quantity:

aptx-4869 - quantity: 112
bandaid - quantity: 131
centrum - quantity: 124
dettol - quantity: 130
gatorade - quantity: 168
huggies - quantity: 126
oralb - quantity: 126
panadol - quantity: 210
super_panadol - quantity: 0
tampax - quantity: 126
vicks - quantity: 84
yakult - quantity: 210
```

```
Enter a command, type "help" for more info.
> additem

Enter item id (must not include whitespace):
> super_panadol

Enter item description:
> extra strength panadol

Item added successfully.


Enter a command, type "help" for more info.
>
```

This image shows the **additem function**, user can enter command **additem / ai** to use additem function. Then, users require to enter the item id (means the item name) and item description. Item will be added into inventory successfully.

```
Enter a command, type "help" for more info.
> addbatch

Enter item id (must not include whitespace):
> super_panadol

Enter stock amount
> 100

Provide the expiry date separately.
Enter the day of expiry date:
> 31

Enter the month of expiry date:
> 12

Enter the year of expiry date:
> 2027

Enter batch description:
> testing

New batch added successfully.


Enter a command, type "help" for more info.
>
```

```
Enter a command, type "help" for more info.
> listbatch

Enter item id (must not include whitespace):
> super_panadol


Batches are listed below as batch ID, date added (descending) and description:

b0 - date added: 21-1-2026, desc: testing


Enter a command, type "help" for more info.
>
```

This image shows the **addbatch function** that is used to add item's batch stocks. User can enter command **addbatch / ab** to use addbatch function. After called the function, user needs to insert the item id, stock amount, expiry date, month, year and description.

```
Enter a command, type "help" for more info.
> viewitem

Enter item id (must not include whitespace):
> super_panadol

Item ID: super_panadol
Description: extra strength panadol
Total stock: 100
Expired stock: 0



Enter a command, type "help" for more info.
>
```

```
Enter a command, type "help" for more info.
> viewitem

Enter item id (must not include whitespace):
> super_panadol

Item ID: super_panadol
Description: extra strength plus++ panadol
Total stock: 100
Expired stock: 0



Enter a command, type "help" for more info.
>
```

```
Enter a command, type "help" for more info.
> edititemdesc

Enter item id (must not include whitespace):
> super_panadol

Enter new description:
> extra strength plus++ panadol

Item description updated.


Enter a command, type "help" for more info.
>
```

This image shows the **edititemdesc function**, it is used to edit item's description. User can enter **edititemdesc** to use edititemdesc function. User will allow to apply a new description for the specific item.

```
Enter a command, type "help" for more info.
> viewbatch

Enter item id (must not include whitespace):
> super_panadol

Enter batch id
> b0

Batch ID: b0
Description: testing
Added: 21-1-2026
Expiry: 31-12-2027
Stock: 100



Enter a command, type "help" for more info.
>
```

```
Enter a command, type "help" for more info.
> viewbatch

Enter item id (must not include whitespace):
> super_panadol

Enter batch id
> b0

Batch ID: b0
Description: MMU student need it
Added: 21-1-2026
Expiry: 31-12-2027
Stock: 100



Enter a command, type "help" for more info.
>
```

```
Enter a command, type "help" for more info.
> editbatchdesc

Enter item id (must not include whitespace):
> super_panadol

Enter batch id (must not include whitespace):
> b0

Enter new batch description:
> MMU student need it

Batch description updated.


Enter a command, type "help" for more info.
>
```

This image shows the **editbatchdesc function** that use to edit the **item's batch description**. User can enter **edititemdesc** to use this function. Then, user needs to insert the item id and the batch id that they want to modify, then enter the new description for that batch.

```
Enter a command, type "help" for more info.
> editbatchstock

Enter item id (must not include whitespace):
> super_panadol

Enter batch id (must not include whitespace):
> b0

Enter transaction type (number 1-4):
1 = SOLD
2 = REFUND
3 = RESTOCK
4 = DISCARD
> 1

Enter sold/discard amount:
> 20

Batch stock updated.
```

This image shows the **editbatchstock function** that use to edit item's batchstock. Users can enter **editbatchstock** to use this function. Then, user needs to insert the item id, batch id, select transaction type (1/2/3/4) and the amount.

```
Enter a command, type "help" for more info.
> viewbatch

Enter item id (must not include whitespace):
> super_panadol

Enter batch id
> b0

Batch ID: b0
Description: MMU student need it
Added: 21-1-2026
Expiry: 31-12-2027
Stock: 100



Enter a command, type "help" for more info.
>
```

Image above is the original batch data **before we use editbatchstock** function:

```
Enter a command, type "help" for more info.
> editbatchstock

Enter item id (must not include whitespace):
> super_panadol

Enter batch id (must not include whitespace):
> b0

Enter transaction type (number 1-4):
1 = SOLD
2 = REFUND
3 = RESTOCK
4 = DISCARD
> 1

Enter sold/discard amount:
> 20

Batch stock updated.
```

```
Enter a command, type "help" for more info.
> viewbatch

Enter item id (must not include whitespace):
> super_panadol

Enter batch id
> b0

Batch ID: b0
Description: MMU student need it
Added: 21-1-2026
Expiry: 31-12-2027
Stock: 80



Enter a command, type "help" for more info.
>
```

When **enter 1** that represents **SOLD**. The stock for the batch will **decrease**.

```
Enter a command, type "help" for more info.
> editbatchstock

Enter item id (must not include whitespace):
> super_panadol

Enter batch id (must not include whitespace):
> b0

Enter transaction type (number 1-4):
1 = SOLD
2 = REFUND
3 = RESTOCK
4 = DISCARD
> 2

Enter refund/restock amount
> 10

Batch stock updated.
```

```
Enter a command, type "help" for more info.
> viewbatch

Enter item id (must not include whitespace):
> super_panadol

Enter batch id
> b0

Batch ID: b0
Description: MMU student need it
Added: 21-1-2026
Expiry: 31-12-2027
Stock: 90



Enter a command, type "help" for more info.
>
```

Continue after enter 1, when **enter 2** that represents **REFUND**. The stock will **increase**.

```
Enter a command, type "help" for more info.
> editbatchstock

Enter item id (must not include whitespace):
> super_panadol

Enter batch id (must not include whitespace):
> b0

Enter transaction type (number 1-4):
1 = SOLD
2 = REFUND
3 = RESTOCK
4 = DISCARD
> 3

Enter refund/restock amount
> 60

Batch stock updated.
```

```
Enter a command, type "help" for more info.
> viewbatch

Enter item id (must not include whitespace):
> super_panadol

Enter batch id
> b0

Batch ID: b0
Description: MMU student need it
Added: 21-1-2026
Expiry: 31-12-2027
Stock: 150


Enter a command, type "help" for more info.
>
```

Continue after enter 2, when enter 3 that repersents **RESTOCK**. The stock will **increase**.

```
Enter a command, type "help" for more info.
> editbatchstock

Enter item id (must not include whitespace):
> super_panadol

Enter batch id (must not include whitespace):
> b0

Enter transaction type (number 1-4):
1 = SOLD
2 = REFUND
3 = RESTOCK
4 = DISCARD
> 4

Enter sold/discard amount:
> 50

Batch stock updated.
```

```
Enter a command, type "help" for more info.
> viewbatch

Enter item id (must not include whitespace):
> super_panadol

Enter batch id
> b0

Batch ID: b0
Description: MMU student need it
Added: 21-1-2026
Expiry: 31-12-2027
Stock: 100


Enter a command, type "help" for more info.
>
```

Continue after enter 3, when enter 4 that represents **DISCARD**. The stock will **decrease**.

```
Enter a command, type "help" for more info.
> listitem

Items are listed below as item ID (ascending) and total stock quantity:

aptx-4869 - quantity: 132
bandaid - quantity: 131
centrum - quantity: 124
dettol - quantity: 130
gatorade - quantity: 168
huggies - quantity: 126
oralb - quantity: 126
panadol - quantity: 210
super_panadol - quantity: 100
tampax - quantity: 126
vicks - quantity: 84
yakult - quantity: 210


Enter a command, type "help" for more info.
>
```

```
Enter a command, type "help" for more info.
> listitem

Items are listed below as item ID (ascending) and total stock quantity:

aptx-4869 - quantity: 132
centrum - quantity: 124
dettol - quantity: 130
gatorade - quantity: 168
huggies - quantity: 126
oralb - quantity: 126
panadol - quantity: 210
super_panadol - quantity: 100
tampax - quantity: 126
vicks - quantity: 84
yakult - quantity: 210


Enter a command, type "help" for more info.
>
```

```
Enter a command, type "help" for more info.
> deleteitem

Enter item id (must not include whitespace):
> bandaid

Item deleted.
```

This image shows the **deleteitem function**. User can enter **deleteitem / di** to use this function. User require to enter the name of the item that they want to delete.

```
Enter a command, type "help" for more info.
> listbatch

Enter item id (must not include whitespace):
> aptx-4869


Batches are listed below as batch ID, date added (descending) and description:

b4 - date added: 21-1-2026, desc: 91Gin
b3 - date added: 21-1-2026, desc: 666
b2 - date added: 21-1-2026, desc: konan


Enter a command, type "help" for more info.
>
```

```
Enter a command, type "help" for more info.
> listbatch

Enter item id (must not include whitespace):
> aptx-4869


Batches are listed below as batch ID, date added (descending) and description:

b3 - date added: 21-1-2026, desc: 666
b2 - date added: 21-1-2026, desc: konan


Enter a command, type "help" for more info.
>
```

```
Enter a command, type "help" for more info.
> deletebatch

Enter item id (must not include whitespace):
> aptx-4869

Enter batch id (must not include whitespace):
> b4

Batch deleted.


Enter a command, type "help" for more info.
>
```

This image shows the **deletebatch function** that use to delete item's batch. User can enter **deletebatch / db** to use this function. User require insert the item id and the batch id that they want to delete.

```
Enter a command, type "help" for more info.
> listitem

Items are listed below as item ID (ascending) and total stock quantity:

aptx-4869 - quantity: 132
centrum - quantity: 124
dettol - quantity: 130
gatorade - quantity: 168
huggies - quantity: 126
oralb - quantity: 126
panadol - quantity: 210
super_panadol - quantity: 100
tampax - quantity: 126
vicks - quantity: 84
yakult - quantity: 210


Enter a command, type "help" for more info.
>
```

This image shows the **listitem function** that allow user to **view the list of the item and total stock** that store inside inventory. User can enter **listitem / li** to use this function.

```
Enter a command, type "help" for more info.
> listbatch

Enter item id (must not include whitespace):
> aptx-4869


Batches are listed below as batch ID, date added (descending) and description:

b3 - date added: 21-1-2026, desc: 666
b2 - date added: 21-1-2026, desc: konan


Enter a command, type "help" for more info.
>
```

This image shows the **listbatch function** that is used to show the **list of batches for the specific item**. Users can enter **listbatch / lb** to use this function.

```
Enter a command, type "help" for more info.
> listtransaction

Transactions are listed below as transaction ID, type,
item id, batch id, transaction amount and date added (descending):

t0 - Type: RESTOCK, Item: advil, Batch: b0, Change: 130, Date: 20-1-2026
t1 - Type: RESTOCK, Item: bandaid, Batch: b0, Change: 60, Date: 20-1-2026
t2 - Type: RESTOCK, Item: bandaid, Batch: b1, Change: 50, Date: 20-1-2026
t3 - Type: RESTOCK, Item: bandaid, Batch: b2, Change: 70, Date: 20-1-2026
t4 - Type: RESTOCK, Item: bandaid, Batch: b3, Change: 30, Date: 20-1-2026
t5 - Type: RESTOCK, Item: centrum, Batch: b0, Change: 124, Date: 20-1-2026
t6 - Type: RESTOCK, Item: dettol, Batch: b0, Change: 65, Date: 20-1-2026
t7 - Type: RESTOCK, Item: dettol, Batch: b1, Change: 65, Date: 20-1-2026
t8 - Type: RESTOCK, Item: gatorade, Batch: b0, Change: 42, Date: 20-1-2026
```

```
t45 - Type: SOLD, Item: super_panadol, Batch: b0, Change: 80, Date: 21-1-2026
t46 - Type: REFUND, Item: super_panadol, Batch: b0, Change: 90, Date: 21-1-2026
t47 - Type: RESTOCK, Item: super_panadol, Batch: b0, Change: 150, Date: 21-1-2026
t48 - Type: DISCARD, Item: super_panadol, Batch: b0, Change: 100, Date: 21-1-2026
```

This image shows the **listtransaction function** that displays the **stock in-out record** in the inventory. Users can enter **listtransaction** to use this function.

```
Enter a command, type "help" for more info.
> viewitem

Enter item id (must not include whitespace):
> super_panadol

Item ID: super_panadol
Description: extra strength plus++ panadol
Total stock: 100
Expired stock: 0



Enter a command, type "help" for more info.
>
```

This image shows the **viewitem function** that allows users to **view item in more detail**. Users can enter **viewitem / vi** to use this function.

```
Enter a command, type "help" for more info.
> viewbatch

Enter item id (must not include whitespace):
> aptx-4869

Enter batch id
> b2

Batch ID: b2
Description: konan
Added: 21-1-2026
Expiry: 1-9-1985
Stock: 100



Enter a command, type "help" for more info.
>
```

This image shows the **viewbatch function** that allows users to view item's batch in more detail. Users can enter **viewbatch / vb** to use this function.

```
Enter a command, type "help" for more info.
> report

Inventory Report:
- panadol | Sold: 0 | Refund: 0 | Batches: 5 | Total: 210 | Expired: 42
- vicks | Sold: 0 | Refund: 0 | Batches: 2 | Total: 84 | Expired: 0
- yakult | Sold: 0 | Refund: 0 | Batches: 5 | Total: 210 | Expired: 42
- centrum | Sold: 0 | Refund: 0 | Batches: 1 | Total: 124 | Expired: 0
- dettol | Sold: 0 | Refund: 0 | Batches: 2 | Total: 130 | Expired: 65
- oralb | Sold: 0 | Refund: 0 | Batches: 3 | Total: 126 | Expired: 42
- tampax | Sold: 0 | Refund: 0 | Batches: 3 | Total: 126 | Expired: 0
- huggies | Sold: 0 | Refund: 0 | Batches: 3 | Total: 126 | Expired: 0
- gatorade | Sold: 0 | Refund: 0 | Batches: 4 | Total: 168 | Expired: 42
- aptx-4869 | Sold: 0 | Refund: 22 | Batches: 2 | Total: 110 | Expired: 110
- super_panadol | Sold: 80 | Refund: 90 | Batches: 1 | Total: 100 | Expired: 0



Enter a command, type "help" for more info.
> |
```

This image shows the **report function** that displays the **inventory report** about sold, sale, batch, total and expired amount to user. Users can enter **report / r** to use this function.

```
Enter a command, type "help" for more info.
> save

Inventory saved to: pims.inventory


Enter a command, type "help" for more info.
> s

Inventory saved to: pims.inventory


Enter a command, type "help" for more info.
> |
```

This image shows the **save function** that will **save change into inventory file**. Users can enter **save / s** to use this function. After saving function, current version will be the latest version and be used next time execution of program.

```
Enter a command, type "help" for more info.
> load

Inventory read from: pims.inventory


Enter a command, type "help" for more info.
> l

Inventory read from: pims.inventory


Enter a command, type "help" for more info.
> |
```

This image shows the **load function** that can **load again the inventory file manually** during program execution. Users can enter **load / l** to use this function. It can be used when users don't want to save the changes that are not ideal, they can use this function to load again inventory file to overwrite the unexpected change and redo again.

```
Enter a command, type "help" for more info.
> exit

Exiting program.



Process returned 0 (0x0)   execution time : 8013.020 s
Press any key to continue.
```

This image shows the **exit function** that is used to exit program. Users can enter **exit / end** to exit program.

CONCLUSION

Our Pharmacy Inventory Management System successfully demonstrates how basic and fundamental data structures and algorithms can be used to solve our problem statements. By implementing linked lists for dynamic inventory storage, queues for transaction tracking, and algorithms such as linear search and selection sort, the system achieves efficient handling of medicine records, batch management, and transaction history.

Choosing the appropriate data structures to match system requirements, ensuring flexibility, accuracy and reliability in managing inventory is shown to be important in system planning. The use of Abstract Data Types (ADTs) provided a clear structure for organizing items, batches, transactions and inventory, making the system modular and easier to maintain.

Our current system design is suitable for small to medium size pharmacy inventories, but future feature additions and development could include integrating advanced search and sorting algorithms, database connectivity and a proper graphical user interface (GUI) to improve our system's usability and scalability.

In conclusion, this project not only inprove our understanding of data structures and algorithms, but also demonstrates their importance in developing robust and reliable software.

TASK DECLARATION FORM

Task declaration form starts at the next page due to table element sizing issues.

Task Declaration Form

Each group member, including the group leader, must individually complete and sign this form.
The signed forms (from all members) are to be appended to the final report for submission.

Pharmacy Inventory Management System

by

Lab 2A Group 2

Group Member's Name	:	Cheong Kai Jin
Group Member's ID	:	241UT240K7

For the purpose of completing this project, I have performed the following tasks:

Coding for user-input, ADT and data structure classes and file save loading functions. Writing final ADT specifications and conclusion.

1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.

2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.

Group Member's Signature	:	*Cheong*
Group Member's Name	:	Cheong Kai Jin
Group Member's ID	:	241UT240K7
Date	:	22/1/2026

Task Declaration Form

Each group member, including the group leader, must individually complete and sign this form.
The signed forms (from all members) are to be appended to the final report for submission.

Pharmacy Inventory Management System

by

Lab 2A Group 2

Group Member's Name	:	Kang Xuan Xin
Group Member's ID	:	241UT240W1

For the purpose of completing this project, I have performed the following tasks:

Coding for add, delete functions for both items and batches, and GUI. Writing the algorithms implementation details and provide program screenshots.

1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.

2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.

Group Member's Signature	:	Xin
Group Member's Name	:	Kang Xuan Xin
Group Member's ID	:	241UT240W1
Date	:	22/1/2026

Task Declaration Form

Each group member, including the group leader, must individually complete and sign this form.
The signed forms (from all members) are to be appended to the final report for submission.

Pharmacy Inventory Management System

by

Lab 2A Group 2

Group Member's Name	:	Chong Zheng Cheng
Group Member's ID	:	241UT240LQ

For the purpose of completing this project, I have performed the following tasks:

Coding for report generation and edit functions for items and batches, and list transactions function. Writing the data structures implementation implementation details and provide program screenshots.

1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.

2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.

Group Member's Signature	:	Chong
Group Member's Name	:	Chong Zheng Cheng
Group Member's ID	:	241UT240LQ
Date	:	22/1/2026

Task Declaration Form

Each group member, including the group leader, must individually complete and sign this form.
The signed forms (from all members) are to be appended to the final report for submission.

Pharmacy Inventory Management System

by

Lab 2A Group 2

Group Member's Name	:	Huda Abdalla Elbashir Elmahdi
Group Member's ID	:	243UT243N4

For the purpose of completing this project, I have performed the following tasks:

Coding for the view, list items and list batch functions. Writing the introduction, problem statement and objectives.

1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.
2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.

Group Member's Signature	:	
Group Member's Name	:	Huda Abdalla Elbashir Elmahdi
Group Member's ID	:	243UT243N4
Date	:	22/1/2026

End of Report